#2 Check that the entry isn’t already in the cache.
#3 If it is, we retrieve the node containing all entry’s data.
#4 Update the value stored in the node.
#5 Move the existing node to the front of the queue.
#6 [OPTIONAL] Returns false to flag that no new entry was added. We need to return here, though, to avoid running the rest of the code in the method.
#7 If the entry is not in the cache, we need to check if the cache is already full.
#8 If it is, we need to delete one element from the cache. The implementation of this method defines the eviction policy of the cache: for LRUs, the oldest element will be removed.
#9 At this point, the cache holds at most maxSize-1 elements, so there is certainly room for one more. Creates a new node at the front of the list.
#10 Add an entry in the cache table pointing at the new node.
#11 If the pointer to the tail of the list was null, it means the list was empty, so now the tail should point to the new element.
#12 Returns true to flag that a new entry was added to the cache.

But when we run into a *cache hit*, accessing an element that is indeed stored on the cache, we need to move an existing list element to the front of the list, and we can only do that efficiently if both we can retrieve in constant(-ish[21]) time a pointer to the linked list node for the existing entry (which could be anywhere in the list, for what we know) and we can remove an element from the list in constant time (again, we need a doubly-linked list for this, with an array-based